

Chrome Browser Extension for Phishing Email Detection

1. Introduction

Phishing email detection is not only a classification problem but also a delivery problem: a technically capable model has limited practical value if its results cannot be accessed at the moment users assess a suspicious message. Browser extensions offer a potential bridge between machine-learning services and the webmail environment, enabling contextual extraction, user-triggered analysis and presentation of security advice within an existing workflow. Yet that convenience creates design tensions involving sensitive message content, browser permissions, interface trust and error management.

This review critically examines browser extensions as a deployment interface for phishing email detection. It distinguishes email-oriented natural language processing (NLP) from URL- and website-centred anti-phishing extensions, evaluates the implications of Chrome Manifest V3, and identifies research gaps relevant to the proposed DeBERTa-v3-large gated-hybrid system with SHAP-based explanation. The central argument is that an extension should be evaluated as a socio-technical security interface, rather than merely as a transport mechanism for a model prediction.

2. Background and key concepts

2.1 Phishing email analysis and the browser context

Email phishing exploits deceptive language, impersonation, urgency and requests for sensitive actions. A browser-based interface may encounter sender information, subject text, visible body content, links and page-specific metadata. However, the document object model (DOM) is a rendering structure rather than a standardised email schema: extraction accuracy depends on the webmail application, its layout, dynamic updates, embedded frames and message threading. Consequently, a successful demonstration on a controlled page does not establish compatibility with arbitrary production webmail interfaces.

2.2 Manifest V3 architecture

Chrome extensions are configured through `manifest.json`, which declares capabilities, content scripts, host access and popup resources. Content scripts can inspect or modify matched pages, whereas extension popup pages support user facing interactions. Chrome documents message passing between extension components and distinguishes the privileges of content scripts from those of extension pages (Google Chrome Developers 2024a; Google Chrome Developers n.d.-a). Manifest V3 also imposes restrictions on remotely hosted code and changes the background execution model; a service worker is relevant where background event handling is required but should not be presumed present in every implementation (Google Chrome Developers n.d.-b).

2.3 Two interaction patterns

A toolbar popup requires explicit user action to open the extension and request analysis. An injected in-page button makes the action available in the email reading context. Both can call the same classification endpoint and return the same underlying prediction, but they differ in discoverability, accessibility, interaction overhead and risk of visual

interference with the host page. These are two interfaces to a shared pipeline, not evidence of two separate classifiers.

2.4 Backend and explanation boundary

A client extension can collect email content and forward it to an application programming interface (API) for model inference. For a hybrid transformer classifier, the backend may generate Safe/Phishing outputs and optional explanatory data. SHAP attribution should be interpreted as model-output contributions under a particular explanation setup, not proof that a message is malicious or that an individual token caused the prediction. The browser must preserve this distinction when presenting confidence and explanatory factors.

2.5 Security and privacy boundary

Host permissions, activeTab access and content-script match patterns determine what an extension can access. Chrome recommends limiting privileges and using optional permissions where feasible (Google Chrome Developers 2024b). Extracted email content may contain private correspondence, identifiers or confidential information; transmitting it to a backend creates additional governance requirements for transport protection, retention, logging and consent.

3. Review of existing research

3.1 Browser-based phishing detection

Dandotiya et al (2026) describe a machine-learning-enhanced Chrome extension for real-time identification of phishing websites using URL and visual-page characteristics. The study supports the feasibility of browser-side security intervention but does not establish that the same feature design transfers to email semantics. URL features can be valuable evidence within an email, yet social-engineering intent often depends on language, sender claims and contextual requests not captured by a website-only classifier.

3.2 Accuracy versus intervention cost

Research evaluating AI-driven anti-phishing Chrome extensions identifies an important trade-off: systems with aggressive detection can also penalise benign content, producing usability costs and unnecessary interruption (Evaluating AI-Driven Heuristics in Chrome-Based Anti-Phishing Extensions 2026). Although that study concerns website-oriented extensions, its broader evaluation insight applies to email tools: high detection rates should be considered alongside false positives, user trust and the consequences of warning fatigue. An interface that treats every model output as a definitive threat risks encouraging uncritical acceptance of uncertain predictions.

3.3 Extension privileges and sensitive data

An empirical security analysis published at the ACM Web Conference reports substantial opportunities for browser extensions to access sensitive information on visited pages (Experimental Security Analysis of Sensitive Data Access by Browser Extensions 2024). This evidence challenges the assumption that a defensive purpose automatically makes an extension low risk. For email-analysis systems, least-privilege access, narrowly

scoped DOM extraction and explicit treatment of transmitted content are integral to the security claim, not optional deployment details.

3.4 Platform architecture as design evidence

Official Chrome documentation establishes the technical roles of manifests, content scripts, message passing and permissions (Google Chrome Developers n.d.-a; 2024b). These sources are authoritative for platform behaviour but are not empirical evidence that any particular phishing detector improves users' decision-making. Similarly, successful extension installation or a working popup is functional evidence, not a substitute for usability, security or classification evaluation.

3.5 Explainability at the point of decision

Explainable machine learning provides a rationale for supplementing binary labels with feature contributions. Lundberg and Lee (2017) formalise SHAP as an additive feature-attribution framework; however, explanations are conditional on the selected model output, reference distribution and representation. Presenting “supporting” and “opposing” factors within a browser extension may improve transparency, but comprehension and behavioural impact require direct user testing. Explanation presence should not be equated with explanation usefulness.

4. Comparison of existing methods or solutions

Four deployment patterns can be distinguished. First, URL or page feature extensions are closely aligned with browser navigation and can intervene at link-visit time, but they may overlook linguistic manipulation inside an email. Second, email content NLP services can represent contextual language more richly but require reliable extraction and potentially transmit sensitive text to a server. Third, rule based email warnings are relatively transparent and inexpensive yet struggle with variation and evasive wording. Fourth, an NLP backed browser extension with post-hoc explanation combines contextual analysis with an accessible interface, but inherits the model's errors and adds security, latency and interpretation requirements.

The popup and injected button patterns are complementary rather than competing algorithms. A popup offers a stable extension owned display surface and explicit activation. An injected button reduces navigation steps but depends more strongly on the host page DOM and can be affected by site redesigns. Neither pattern guarantees better detection accuracy because both may submit the same extracted input to the same backend. The appropriate comparison concerns interaction success, extraction consistency, task completion time and user comprehension.

For backend placement, fully local inference can reduce content transfer but may be constrained by browser resources and large transformer model size. Remote inference accommodates a computationally intensive model and centralised updates, but introduces availability, network latency, confidentiality and operational costs. A rigorous comparison therefore includes end-to-end latency, failure modes, privacy exposure and maintainability rather than model accuracy alone.

5. Research gaps and limitations

5.1 Email centered evidence gap

Much extension focused anti-phishing literature concentrates on URLs or deceptive websites. The effectiveness of browser-based analysis of rendered email text, especially when paired with a large contextual transformer, remains a distinct question. Claims should not be transferred uncritically from website classification to email classification.

5.2 Extraction robustness

Controlled HTML test pages provide repeatable input but may not reproduce dynamic production webmail structures, message threads, collapsed content, inline attachments, internationalised text or changing selectors. Evaluation should separate successful extraction from successful classification and report failures independently.

5.3 Privacy and trust

An extension can observe information users did not intend to share with an external service. Published extension-security research makes permission scope and data handling a first-order research issue. Prototype documentation should identify the trust boundary, transmitted fields, backend destination, error handling and whether email data are stored.

5.4 Human factors

A Safe label can be mistaken for a guarantee, while a Phishing label can be mistaken for certainty. Confidence is model reported rather than necessarily calibrated. SHAP feature contributions may be misunderstood as causal evidence or as percentages. Few functional demonstrations test whether users correctly interpret supporting and opposing factors.

5.5 Operational evidence

Demonstrating popup and injected-button flows does not by itself establish cross-site compatibility, adversarial resilience, accessibility, scalability or real-world effectiveness. Tests should distinguish expected outputs, observed outputs, ground-truth labels and any disagreements. False-positive analysis and response-time measurement are particularly important for user-facing warnings.

6. Implications for the proposed project

6.1 Architecture and scope

The proposed prototype uses a Chrome Manifest V3 extension with a toolbar popup (“Analyse Current Email”) and an injected “Scan Email for Phishing” control. Both should be documented as interaction routes into a shared email-analysis pipeline connected to the FastAPI backend and DeBERTa-v3-large gated-hybrid classifier. This framing avoids overstating the independence of the two workflows.

6.2 Functional evaluation

Testing should record whether the correct message was selected; whether sender, subject and body were extracted accurately; whether the backend request succeeded; whether the returned label and confidence were displayed consistently; and whether the

SHAP sections correspond to the same analyzed email. Controlled Safe and Phishing cases should be supplemented with empty content, missing fields, long emails, network errors and unexpected page layouts.

6.3 Explanation design

Where SHAP outputs are exposed, the interface should label the explained class, separate supporting from opposing contributions, and rank strongest factors by absolute magnitude only if that is how the implementation computes them. A concise note should clarify that SHAP contributions are not probabilities or causal proof. The display must not silently substitute heuristic indicators for actual SHAP values.

6.4 Permission and privacy discipline

The extension should request only necessary permissions, restrict matching hosts where practical, avoid transmitting unrelated page content and use protected communication appropriate to deployment. Any production claim requires a documented assessment of storage, logging, authentication, transport security and consent. These recommendations are design implications, not verified characteristics of the current prototype.

6.5 Evidence and claims

The project's controlled email testing documents can demonstrate that the popup and injected button invoke classification in a reproducible environment. They are prototype functional validation, not proof of universal Gmail support or a field tested commercial extension. Future evaluation should measure extraction accuracy, false positives, response latency, user comprehension and reliability across browser and webmail changes.

7. Conclusion

Browser extensions offer a practical mechanism for bringing phishing email analysis into the user's reading workflow, but their value depends on more than a functioning classifier. The reviewed literature supports the feasibility of browser-based phishing intervention while revealing limitations in transferring website-oriented evidence to email-language analysis. It also demonstrates that extension privileges, sensitive-data access, false alarms and explanation interpretation must be treated as substantive research concerns.

For the proposed project, the defensible contribution is an integrated prototype that exposes a shared NLP detection and explanation pipeline through two browser interaction mechanisms. A high-quality evaluation should establish what has actually been demonstrated controlled email extraction, backend communication, result display and explanation presentation while treating production compatibility, privacy assurance and behavioural effectiveness as further research and engineering objectives.

References

- Dandotiya, M, Goyal, NK, Khunteta, A & Tiwari, B 2026, 'Real time identification of phishing attacks through machine learning enhanced browser extensions', Scientific Reports, vol. 16, article 6612, <https://doi.org/10.1038/s41598-026-35655-7>
- 'Evaluating AI-Driven Heuristics in Chrome-Based Anti-Phishing Extensions' 2026, Procedia Computer Science, vol. 283, pp. 32–41, <https://doi.org/10.1016/j.procs.2026.06.092>
- 'Experimental Security Analysis of Sensitive Data Access by Browser Extensions' 2024, Proceedings of the ACM Web Conference 2024, pp. 1283–1294, <https://doi.org/10.1145/3589334.3645683>
- Google Chrome Developers 2024a, 'Extensions architecture overview', Chrome for Developers, <https://developer.chrome.com/docs/extensions/develop/concepts/architecture>
- Google Chrome Developers 2024b, 'Declare permissions', Chrome for Developers, <https://developer.chrome.com/docs/extensions/develop/concepts/declare-permissions>
- Google Chrome Developers n.d.-a, 'Content scripts', Chrome for Developers, <https://developer.chrome.com/docs/extensions/develop/concepts/content-scripts>
- Google Chrome Developers n.d.-b, 'Migrate to a service worker', Chrome for Developers, <https://developer.chrome.com/docs/extensions/develop/migrate/to-service-workers>
- Lundberg, SM & Lee, S-I 2017, 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, vol. 30, <https://arxiv.org/abs/1705.07874>